

Python Code

```
1  import os
2  import time
3  import logging
4  import yaml
5  import subprocess
6  from azure.ai.ml import MLClient, Input
7  from azure.ai.ml.constants import AssetTypes
8  from azure.ai.ml.entities import Data
9  from azure.identity import DefaultAzureCredential
10
11 # Configure clean console logging
12 logging.basicConfig(level=logging.INFO, format='%(asctime)s %(levelname)s %(message)s')
13 log = logging.getLogger(__name__)
14
15 # --- Configuration & Constants ---
16 RESOURCE_GROUP = "rg-uk-realestate-analytics"
17 COMPUTE_NAME = "RealEstateML"
18 EXPERIMENT_NAME = "housing-sampled-v3"
19
20 # Data structural definitions
21 DATA_DIR_PATH = "./lean_production_parquet"
22 TARGET_COLUMN = "Log_Price"
23 PRIMARY_METRIC = "r2_score"
24
25 def main():
26     # 1. Create the MLTable control file programmatically
27     log.info("Generating data stream architecture layout...")
28     mltable_content = """
29 paths:
30   - pattern: ./*.parquet
31 transformations:
32   - read_parquet: {}
33 """
34     mltable_path = os.path.join(DATA_DIR_PATH, "MLTable")
35     with open(mltable_path, "w") as f:
36         f.write(mltable_content.strip())
37
38     # 2. Authenticate and Register Cloud Data Asset
39     log.info("Authenticating with Azure Environment...")
40     credential = DefaultAzureCredential()
41     client = MLClient.from_config(credential=credential)
42
43     data_version = str(int(time.time()))
44     data_meta = Data(
45         name="dataset_uk_housing_log_transformed",
46         version=data_version,
47         description="Streamed log-scale UK housing data asset layout",
48         path=DATA_DIR_PATH,
49         type=AssetTypes.MLTABLE
```

```

50     )
51
52     log.info(f"Registering fresh cloud asset version {data_version}...")
53     uploaded_asset = client.data.create_or_update(data_meta)
54
55     # 3. Create raw YAML configuration tracking manifest
56     yaml_manifest = {
57         "$schema":
58         "https://azuremlschemas.azureedge.net/latest/autoMLJob.schema.json",
59         "type": "automl",
60         "task": "regression",
61         "primary_metric": PRIMARY_METRIC,
62         "experiment_name": EXPERIMENT_NAME,
63         "compute": COMPUTE_NAME,
64         "training_data": {
65             "type": "mltable",
66             "path": uploaded_asset.id
67         },
68         "target_column_name": TARGET_COLUMN,
69         "limits": {
70             "timeout_minutes": 120,
71             "trial_timeout_minutes": 15,
72             "max_trials": 25,
73             "max_concurrent_trials": 4,
74             "enable_early_termination": True
75         },
76         "training": {
77             "enable_model_explainability": True,
78             "blocked_training_algorithms": ["RandomForest",
79             "ExtremeRandomTrees"],
80             "enable_vote_ensemble": True,
81             "enable_stack_ensemble": True
82         },
83         "featurization": {
84             "mode": "auto",
85             # FIXED: Aligned strictly to the 'imputer' keyword set accepted by
86             # your schema environment.
87             # This turns off standard/minmax scaling modifications for all
88             # numerical feature vectors
89             # while smoothly leaving categorical and time featurization
90             # active.
91             "transformer_params": {
92                 "imputer": [
93                     {
94                         "fields": ["_all_numeric_"],
95                         "parameters": {
96                             "strategy": "mean"
97                         }
98                     }
99                 ]
100             }
101         },
102         "n_cross_validations": 2
103     }

```

```
99
100 yaml_path = "automl_job_config.yaml"
101 with open(yaml_path, "w") as f:
102     yaml.dump(yaml_manifest, f, default_flow_style=False)
103 log.info(f"Generated standalone deployment manifest: {yaml_path}")
104
105 # 4. Hand execution tracking off directly to Azure ML CLI v2 engine
106 log.info("Submitting run tracking layer to Azure Foundry via CLI...")
107 cli_command = f"az ml job create --file {yaml_path} --resource-group
{RESOURCE_GROUP} --workspace-name {client.workspace_name}"
108
109 result = subprocess.run(cli_command, shell=True, capture_output=True,
text=True)
110
111 if result.returncode == 0:
112     print("\n" + "="*80)
113     print(" Job successfully submitted via CLI engine layout wrapper!")
114     print(result.stdout)
115     print("="*80 + "\n")
116 else:
117     print("\n" + "!"*80)
118     print(" CLI Submission Error:")
119     print(result.stderr)
120     print("!"*80 + "\n")
121
122 if __name__ == '__main__':
123     main()
124
```